TC5061 - Operaciones de Aprendizaje Automático (MLOps)

Actividad Semana 4: Código de Experimentación Reutilizable Instrumentado con MLFlow

Estudiante:	Jesus Ricardo Bahamon Quimbaya (Matrícula: A01840023)	**Profesor:**	Dr. Gerardo Reyes
Institución:	Tecnológico de Monterrey - MNA	**Fecha:**	Octubre 2026
Repositorio GitHub:	https://github.com/ricardo-bahamon/TC5061-MLFlow-Experimentation	**Estructura:**	Cookiecutter Data Science

1. Migración Modular y Estructura Cookiecutter Data Science

En esta actividad individual se transformó el flujo experimental y de entrenamiento del modelo base sobre el conjunto de datos *Wine Quality (Red Wine)* en un pipeline de producción modular, reutilizable y parametrizado (train.py). El cuaderno Jupyter original (notebooks/Wine_EDA.ipynb) se reservó exclusivamente para análisis exploratorio y visualización preliminar. Siguiendo rigurosamente la plantilla **Cookiecutter Data Science**, el repositorio organiza el código en módulos con responsabilidades únicas y desacopladas: src/data/make_dataset.py implementa la detección automática de separadores (coma o punto y coma), limpieza y eliminación de duplicados; src/features/build_features.py aísla las covariables y realiza la partición estratificada de entrenamiento y prueba (80/20); y src/models/train_model.py construye un Pipeline de scikit-learn compuesto por StandardScaler y LogisticRegression, evitando fugas de información (*data leakage*).

2. Parametrización de Ejecución con Argumentos de Línea de Comandos

El script train.py implementa una interfaz CLI mediante argparse que expone cuatro hiperparámetros de modelado y la semilla determinista: la fuerza de regularización (--C, tipo float), el algoritmo de optimización (--solver: lbfgs, saga, liblinear), la norma de penalización (--penalty: l1, l2), el número máximo de iteraciones (--max-iter) y la semilla pseudoaleatoria (--seed). Además, permite configurar la ruta de datos, la variable objetivo, el tamaño de prueba y los pliegues de validación cruzada. Cualquier colaborador o evaluador puede reproducir el flujo completo con valores predeterminados sin modificar el código fuente.

Comando exacto de ejecución con valores por defecto:

```
python train.py
```

Comando exacto para la ejecución con hiperparámetros personalizados (Mejor Corrida):

```
python train.py --experiment-name "TC5061_Wine_Quality_Experimentation" --run-name
"run_5_lasso_l1_saga" --C 1.0 --solver saga --penalty l1 --max-iter 1500 --seed 42
```

3. Instrumentación con MLflow Tracking

Cada ejecución del script se gestiona de forma atómica dentro de un contexto mlflow.start_run(), agrupada bajo el experimento común TC5061_Wine_Quality_Experimentation. La instrumentación registra: (a) **Hiperparámetros y metadatos** con mlflow.log_params (C, solver, penalty, max_iter, seed, scaler, dimensiones); (b) **Métricas de evaluación** con mlflow.log_metrics (Accuracy, macro/weighted F1-Score, macro Precision, macro Recall, junto con el promedio y dispersión de exactitud en 5 pliegues de validación cruzada); (c) **Artefactos de diagnóstico** con mlflow.log_artifact (matriz de confusión gráfica en PNG a 300 DPI, reporte textual por clase classification_report.txt y resumen en JSON metrics_summary.json); y (d) **Modelo serializado** con mlflow.sklearn.log_model, guardando el pipeline completo con firma de inferencia, ejemplos de entrada y dependencias para reproducibilidad y despliegue.

4. Comparación Experimental y Análisis de Resultados

Se ejecutó una serie sistemática de seis corridas experimentales en MLflow variando la intensidad de regularización ($C \in \{0.01, 0.1, 1.0, 10.0\}$), la norma de penalización (L1 vs L2) y el optimizador. A continuación se presentan los resultados consolidados registrados y comparados en la interfaz de MLflow Tracking:

Corrida / Nombre	C	Solver	Norma	Seed	Exactitud (Test)	F1 Weighted	CV Exactitud (5-Folds)
Run 1: Baseline L2	1.0	lbfgs	L2	42	0.5809	0.5541	0.5943 ± 0.023
Run 2: Alta Reg. (C=0.01)	0.01	lbfgs	L2	42	0.5662	0.5113	0.5658 ± 0.024
Run 3: Reg. Moderada (C=0.1)	0.1	lbfgs	L2	42	0.5846	0.5549	0.5842 ± 0.021
Run 4: Baja Reg. (C=10.0)	10.0	lbfgs	L2	42	0.5809	0.5571	0.5897 ± 0.027
Run 5: Lasso L1 SAGA (Mejor)	1.0	saga	**L1**	42	**0.5919**	**0.5655**	0.5861 ± 0.024
Run 6: Réplica Determinista	1.0	lbfgs	L2	42	0.5809	0.5541	0.5943 ± 0.023

Identificación de la Mejor Configuración:

La configuración con el desempeño superior en el conjunto de prueba independiente fue la **Corrida 5 (run_5_lasso_l1_saga)**, alcanzando una **Exactitud de 0.5919 (59.19%)** y un **F1-Score ponderado de 0.5655**, superando al modelo base por 1.10 puntos porcentuales en exactitud y 1.14 puntos en F1 ponderado. Este modelo utiliza regularización Lasso (penalización L1 con C=1.0) resuelto mediante el algoritmo de descenso de gradiente estocástico promedio adaptado (saga), permitiendo convergencia estricta en 1500 iteraciones.

Hiperparámetro Más Influyente y Sensibilidad:

El hiperparámetro con mayor impacto estructural fue el **tipo de penalización (penalty: L1 vs L2)**, seguido estrechamente por el inverso de regularización (C). La penalización L1 induce dispersión (*sparsity*) anulando los coeficientes de covariables colineales o con bajo aporte explicativo (como densidad o acidez fija), lo cual atenúa el ruido en las decisiones multiclase entre calidades contiguas (5, 6 y 7). Por otro lado, una regularización desmedida (**C=0.01** en Run 2) provocó un **subajuste severo (underfitting)**, desplomando la exactitud a 0.5662 y el F1 ponderado a 0.5113 debido a que contrajo excesivamente los pesos de variables fisicoquímicas críticas como el alcohol y los sulfatos.

Documentación de Reproducibilidad y Dependencias:

La reproducibilidad científica se verificó empíricamente mediante la **Corrida 6**, la cual reprodujo de manera idéntica los argumentos de la Corrida 1 (C=1.0, lbfgs, L2, seed=42). Como se observa en la tabla, ambas corridas arrojaron métricas exactamente idénticas hasta el último decimal (Exactitud = 0.580882, CV = 0.594326), validando que la fijación de la semilla en el particionado, el escalado y el optimizador garantiza determinismo absoluto. El entorno computacional se encuentra completamente fijado en requirements.txt con versiones exactas (mlflow==3.16.1, scikit-learn==1.7.2, pandas==2.2.3, numpy==2.2.6).

Limitaciones del Modelo:

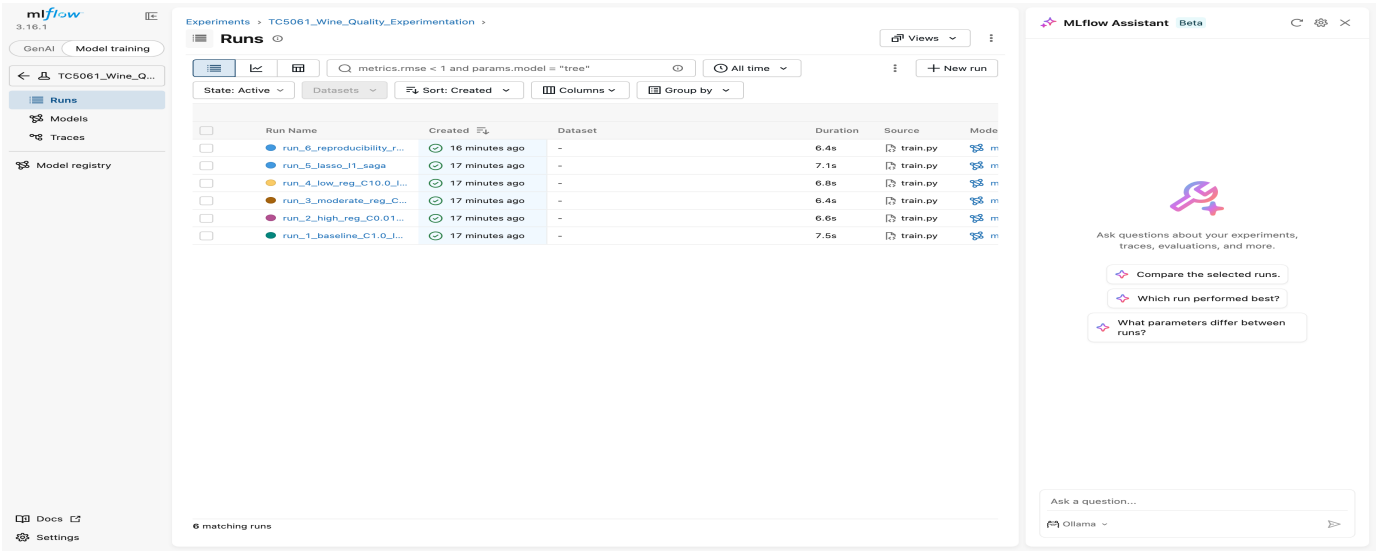
Se identificaron dos limitaciones centrales: (1) **Fuerte desbalance de clases en la calidad del vino**: más del 82% de las muestras corresponden a las clases 5 y 6, mientras que las calificaciones extremas (3 y 8) cuentan con menos de 15 ejemplos en total. Esto provoca que el modelo lineal ignore las clases minoritarias (precisión y recall cercanos a 0 en clases 3 y 8), reflejándose en una brecha notable entre el F1 Macro (0.276) y el F1 Weighted (0.565). (2) **Suposición de linealidad**: Las interacciones organolépticas entre pH, acidez volátil y alcohol presentan fronteras no lineales que una regresión logística no puede capturar sin expansiones polinomiales o árboles de decisión.

Anexo A: Evidencias de Experimentación en MLflow Tracking

Capturas oficiales de la interfaz web de MLflow Tracking demostrando el registro de corridas y comparación visual.

Figura 1: Tabla General de Corridas Registradas en el Experimento MLFlow

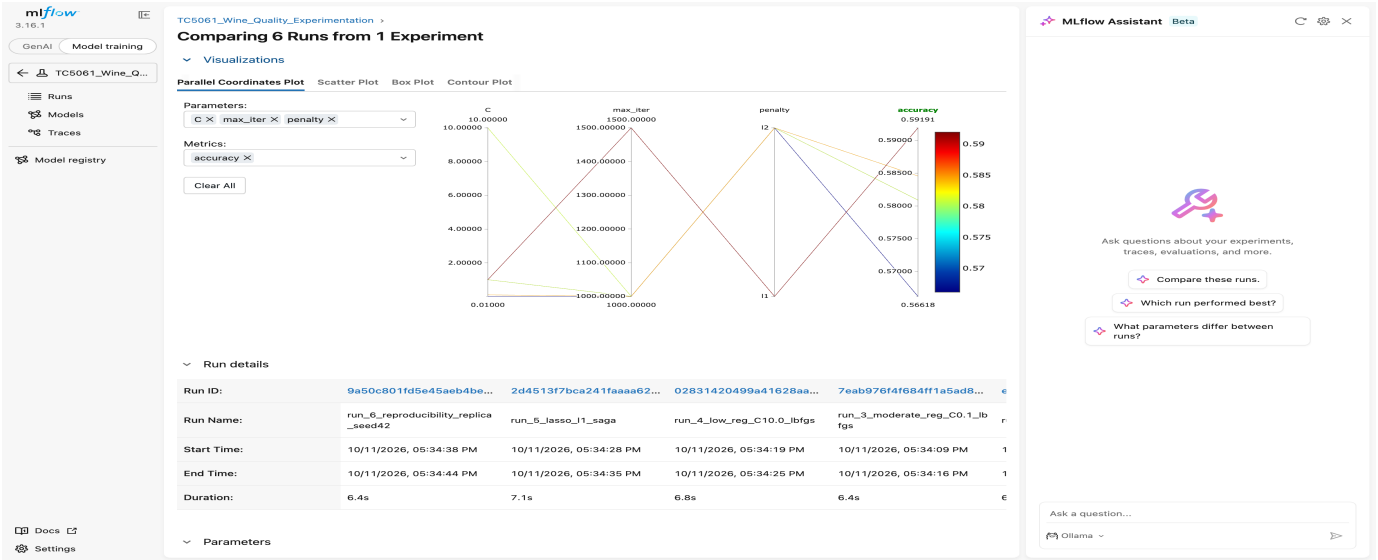
Vista de la pestaña *Model training* en MLflow (puerto 5050), exhibiendo las seis corridas completadas con sus respectivos identificadores, estado, hiperparámetros (C, solver, penalty, seed) y métricas de evaluación.



Captura 1: Vista general de las seis corridas en el experimento 'TC5061_Wine_Quality_Experimentation'.

Figura 2: Vista Comparativa de Corridas y Gráficos de Coordenadas Paralelas

Herramienta interactiva de comparación de MLflow contrastando las configuraciones evaluadas. El gráfico de coordenadas paralelas ilustra el impacto directo del parámetro C y la penalización en el balance entre exactitud y F1-score.



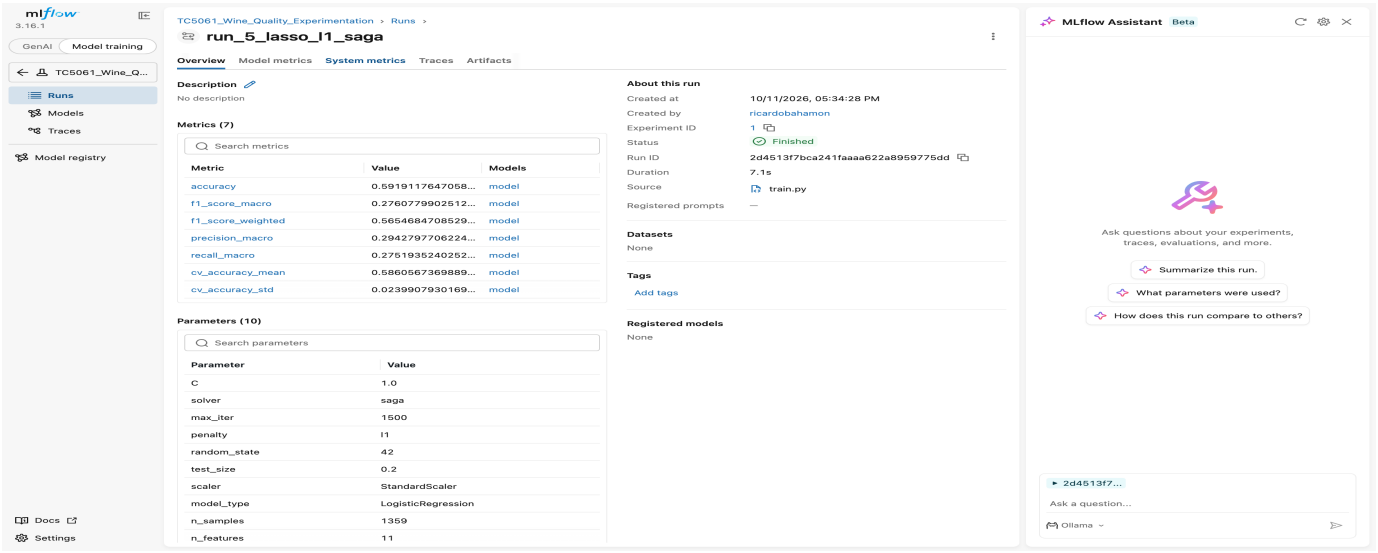
Captura 2: Vista comparativa avanzada en MLflow con gráfico de coordenadas paralelas y contraste de hiperparámetros.

Anexo B: Detalle de Corrida Óptima y Artefactos Registrados

Inspección detallada de la mejor corrida ('run_5_lasso_l1_saga') con sus métricas y artefactos serializados.

Figura 3: Detalle de la Corrida Óptima (run_5_lasso_l1_saga)

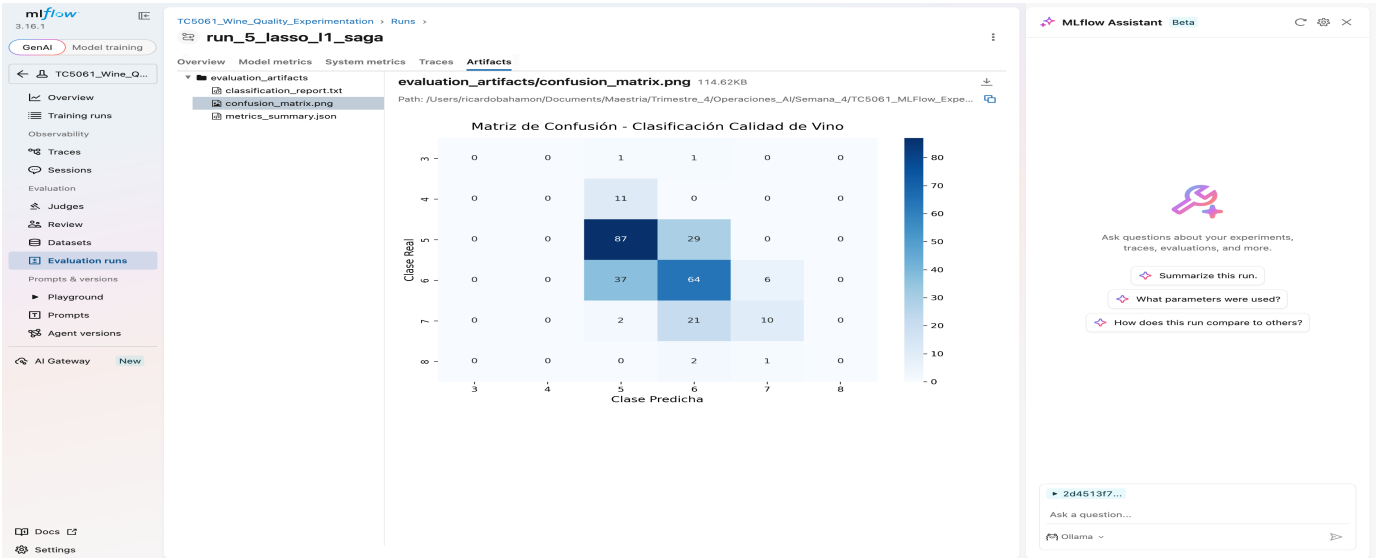
Ficha técnica de la corrida en MLflow mostrando el Run ID (2d4513f7bca241faaaa622a8959775dd), la totalidad de hiperparámetros registrados con log_param y las métricas computadas con log_metric.



Captura 3: Inspección de metadata, hiperparámetros y métricas de la corrida óptima en MLflow.

Figura 4: Explorador de Artefactos con Previsualización de Matriz de Confusión

Pestaña de artefactos en MLflow mostrando el directorio evaluation_artifacts/ desplegado con la matriz de confusión (confusion_matrix.png) renderizada en el visor interactivo, el reporte textual y el resumen de métricas.



Captura 4: Explorador de artefactos en MLflow exhibiendo la matriz de confusión y artefactos de evaluación.